

Complemento Práctico: Arquitectura de Objetos

Dependencias y Calidad

Hasta ahora vimos relaciones "fuertes" donde un objeto tiene a otro (Asociación, Composición). Pero, ¿qué pasa cuando solo necesitamos usar un objeto por un instante? En esta unidad exploramos las Relaciones de Dependencia (flechas punteadas en UML) y las métricas que separan a un programador novato de uno profesional: Cohesión, Acoplamiento y el principio "Tell, Don't Ask".



Contenido

Dependencia de Uso

Relaciones temporales entre objetos que no requieren asociación permanente

Dependencia de Creación

Cómo una clase puede ser responsable de instanciar objetos de otras clases

Cohesión y Acoplamiento

Las métricas fundamentales para evaluar la calidad de tu diseño

Tell, Don't Ask

El principio que transforma código procedimental en verdadero código orientado a objetos

Dependencia de Uso: Relación Temporal

La dependencia de uso ocurre cuando una clase necesita a otra solo para ejecutar un método específico. No la guarda como atributo, la usa y la olvida. Es como cuando pedís prestado un libro: lo usás para una tarea y lo devolvés. No se convierte en parte de tu biblioteca permanente.

Pensemos en el ejemplo clásico de una Impresora y un Documento. La Impresora no tiene un documento como atributo (no es su dueño). Solo lo recibe como parámetro en el método imprimir(), lo procesa y termina. Esta es una relación mucho más débil que la asociación o la composición.

Código Java

```
public class Impresora {  
    // El documento llega,  
    // se usa y desaparece  
    public void imprimir(Documento  
doc) {  
        if (this.encendida) {  
            System.out.println(  
                doc.getTitulo()  
            );  
            System.out.println(  
                doc.getCuerpo()  
            );  
        }  
    }  
}
```

Características clave

- El objeto dependiente no se almacena
- Solo existe durante la ejecución del método
- En UML se representa con línea punteada
- Etiqueta «use» o «call»

Dependencia de Creación: Fabricando Objetos

La dependencia de creación es más fuerte que la de uso porque implica que una clase es responsable de instanciar objetos de otra. No solo usa el objeto: lo crea desde cero. Esto significa que la clase debe conocer el constructor y los detalles de inicialización de la clase dependiente.

Veamos el ejemplo del `GestorUsuario`. Esta clase no recibe un usuario ya creado, sino que recibe un nombre como `String` y dentro del método crea la instancia del `Usuario`. El `Gestor` es el fabricante, tiene la responsabilidad de dar vida al objeto.

```
public class GestorUsuario {  
    public void crearUsuario(String nombre) {  
        // La creación ocurre AQUÍ dentro  
        Usuario u = new Usuario(nombre);  
        System.out.println("Creado: " + u.getNombre());  
    }  
}
```

Dependencia de Uso

Recibe el objeto como parámetro.
Bajo acoplamiento.

Dependencia de Creación

Instancia el objeto internamente.
Mayor acoplamiento.

En UML se representa con una línea punteada y la etiqueta «create». Esta dependencia es más rígida porque cambios en el constructor de `Usuario` afectan directamente al `GestorUsuario`.

Alta Cohesión: Una Sola Responsabilidad

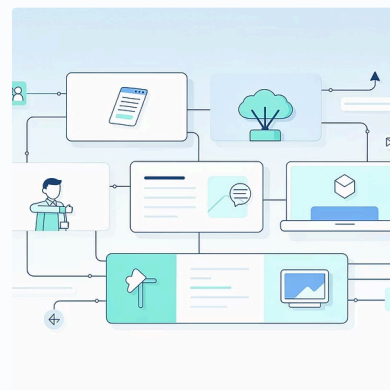
La cohesión mide qué tan enfocada está una clase en una única tarea. Una clase con alta cohesión hace una cosa y la hace bien. Es como un especialista médico: el cardiólogo se dedica al corazón, no hace cirugía cerebral, no atiende fracturas ni receta anteojos.

El problema más común en principiantes es la "Clase Dios": una clase mastodóntica que hace todo. Imaginate una clase Sistema que cobra facturas, envía emails, guarda en la base de datos y calcula impuestos. ¿Qué pasa si querés cambiar cómo se envían los emails? Tenés que tocar una clase que también maneja facturación y persistencia. Es una pesadilla para mantener.



Baja Cohesión (Malo)

Clase Dios que hace todo: facturación, emails, BD, impuestos. Difícil de mantener y testear.



Alta Cohesión (Bien)

Clases separadas: Facturador, NotificadorMail, RepositorioBD. Cada una con responsabilidad única.

La alta cohesión facilita el testing (probás una cosa a la vez), mejora la reutilización (podés usar NotificadorMail en otros proyectos) y simplifica el mantenimiento. Cuando necesitás modificar algo, sabés exactamente dónde ir.

Bajo Acoplamiento: Independencia entre Clases

El acoplamiento mide cuán dependientes son las clases entre sí. Alto acoplamiento significa que cambiar una línea de código en una clase obliga a modificar muchas otras. Es como un efecto dominó: tocás una pieza y se cae todo el sistema.

Si cambiar la clase Motor te obliga a reescribir Auto, Taller y Conductor simultáneamente, tenés un problema serio de alto acoplamiento. El objetivo es diseñar clases que sean lo más independientes posible, de modo que los cambios en una impacten mínimamente en las demás.

¿Cómo lograr bajo acoplamiento?

- Usá interfaces para desacoplar implementaciones concretas
- Preferí dependencias de uso sobre asociaciones permanentes
- Aplicá el principio de inversión de dependencias
- Evitá que las clases conozcan detalles internos de otras
- Usá inyección de dependencias cuando sea posible

Las dependencias de uso (temporales) son menos rígidas que las asociaciones permanentes, ayudando a reducir el acoplamiento. Recordá: bajo acoplamiento no significa cero relaciones, sino relaciones flexibles y bien pensadas.

El Equilibrio: Alta Cohesión + Bajo Acoplamiento

1

Responsabilidad por clase

Cada clase debe hacer una sola cosa excepcionalmente bien

3-5

Dependencias ideales

Una clase debería depender de pocas otras clases

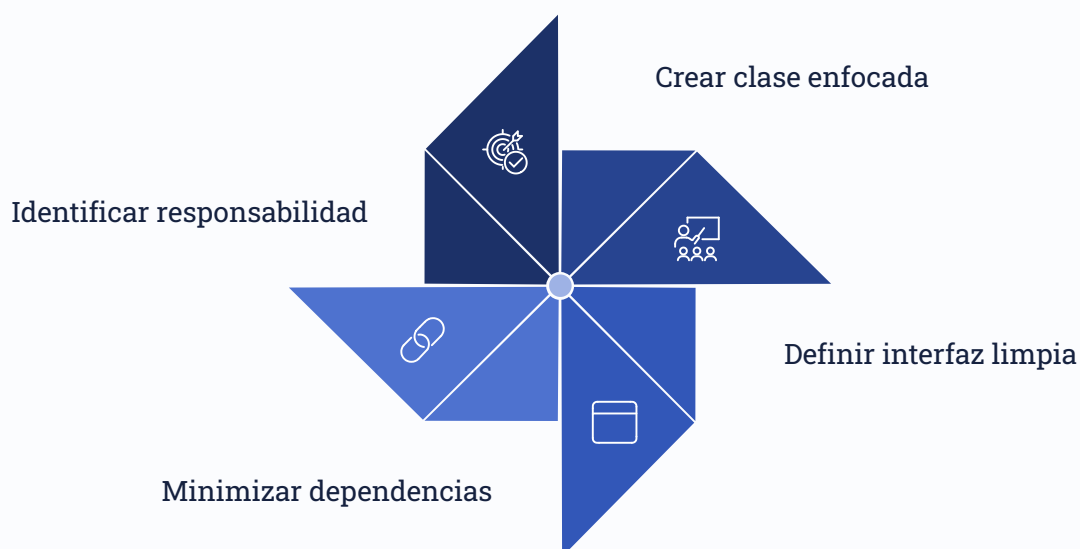
80%

Código reutilizable

Con buen diseño, la mayoría del código puede reutilizarse

El diseño orientado a objetos profesional busca maximizar la cohesión dentro de cada clase mientras minimiza las dependencias entre clases. Estos dos principios trabajan juntos: cuando cada clase tiene una responsabilidad clara y enfocada (alta cohesión), naturalmente necesita menos colaboración con otras clases (bajo acoplamiento).

Pensalo como un equipo de trabajo bien organizado. Cada miembro tiene su especialidad (alta cohesión) y los equipos se comunican a través de interfaces claras sin entrometerse en los detalles internos de otros (bajo acoplamiento). El resultado es un sistema flexible, mantenible y escalable.



El Problema: "Ask" (Preguntar)

Este es el error conceptual más grave al aprender POO: usar objetos como si fueran simples estructuras de datos (Structs en C). El código "pregunta" constantemente al objeto por sus datos internos, toma decisiones afuera y luego setea los resultados. Es como si un mecánico desarmara todo el motor, lo analizara pieza por pieza en su mesa de trabajo, decidiera qué hacer y después volviera a ensamblar todo.

Ejemplo Incorrecto (Código Procedimental Disfrazado)

```
// El Auto le pregunta todo al motor y decide por él
if (motor.getTemperatura() < 100) {
    motor.setCombustible(motor.getCombustible() - 1);
}
```

¿Qué está mal aquí?

El Auto conoce demasiado sobre la lógica interna del Motor: sabe que debe verificar temperatura, sabe cómo calcular el combustible, sabe cuándo inyectar.

Consecuencias

Si la lógica del motor cambia (ahora depende de presión y temperatura), el Auto se rompe. El encapsulamiento es inexistente.

Acoplamiento Extremo

Cada vez que modificás Motor, tenés que buscar todos los lugares donde se hacen estos gets/sets y actualizarlos.

La Solución: "Tell, Don't Ask" (Decile, No Le Preguntes)

En lugar de pedir datos (get) para procesarlos afuera, le decimos al objeto qué queremos lograr (Tell) y dejamos que él decida cómo hacerlo con sus propios datos. Es como pedirle al mecánico que "ajuste el motor" en lugar de darle instrucciones paso a paso sobre cada tornillo que debe girar.

Código Incorrecto (ASK)

```
// Exponemos lógica interna
if (motor.getTemperatura() < 100) {
    motor.setCombustible(
        motor.getCombustible() - 1
    );
}
```

El Auto maneja la lógica que debería estar en Motor.

Código Correcto (TELL)

```
// Delegamos responsabilidad
motor.inyectarCombustible();

// Dentro de Motor:
public void inyectarCombustible() {
    if (this.temperatura < 100) {
        this.combustible--;
    }
}
```

La lógica vive donde están los datos.

Observá cómo en la versión correcta, el Auto simplemente da una orden de alto nivel: "inyectá combustible". El Motor sabe cómo hacerlo porque tiene acceso directo a sus propios atributos (temperatura, combustible) y contiene la lógica de negocio que los gobierna.

Principios Clave para Código Profesional

Llegamos al final de este recorrido por las dependencias y la calidad de diseño. Estos conceptos son fundamentales para escribir código orientado a objetos profesional, no solo código que "funciona".

Usá dependencias sabiamente

Las dependencias de uso son más flexibles que las asociaciones. Úsalas cuando solo necesitás un objeto temporalmente.

Maximizá la cohesión

Cada clase debe tener una responsabilidad única y clara. Evitá las "Clases Dios" que hacen todo.

Minimizá el acoplamiento

Diseñá clases independientes. Los cambios en una clase no deberían obligarte a reescribir muchas otras.

Tell, Don't Ask

No uses getters para exponer lógica interna. Llevá la lógica hacia donde están los datos. Delegá responsabilidades.

"Los getters no son malos per se, pero abusar de ellos para extraer datos y tomar decisiones afuera del objeto destruye el encapsulamiento. Usá getters solo cuando realmente necesitás mostrar un dato, no para implementar lógica."

Aplicando estos principios en tus proyectos de Programación 2, vas a notar la diferencia: código más fácil de mantener, testear y extender.